# *IFT6135: Representation Learning (Generative Models)*

*Matthew He*

*April 5, 2026*

## *Contents*



## 1 *Preliminaries*

While discriminative models (like classifiers) predict labels given data, generative models attempt to learn the underlying true data distribution.

### 1.1 *Motivations*

Generative modeling was once questioned for its immediate utility, i.e., why bother putting research towards building things that replicate examples that are kind of in our data set when we're already struggling to build a model (e.g., classifiers)? Nevertheless, it is now central to advanced AI systems.

Several concrete motivation for generative modeling include:

- **Useful learning signal for semi-supervised learning.** We want to extract out useful features of the generative model and use them to help with downstream tasks.

**Remark 1** (Semi-supervised learning (SSL))**.** In a semi-supervised learning (SSL) setup, we have a tiny dataset of labeled examples (e.g., 100 images of cats and dogs) and a massive dataset of unlabeled examples (e.g., 1,000,000 random images from the internet). The goal of SSL is to somehow use those 1,000,000 unlabeled images to help the model learn.

- **Synthesize new observations–world models.** Generative models can simulate environment dynamics (e.g., Google's Genie), which are useful for planning or training reinforcement learning (RL) agents.
- **As a prior over real observations.** Generative models can be useful for denoising or super-resolution. It can also works as a way of to compress data/information. For example, video compression via dynamic re-synthesis.

## 1.2 *Taxonomy of generative models*

Generative models can be categorized into two main types:

- **Directed graphical models.** These includes autoregressive, normalizing flows, variational autoencoders (VAEs), and diffusion models. They model probability through explicit causal directions, i.e., by conditioning from the top latent representation to observations through a series of prior distributions:

$$p(x, h^1, h^2, h^3) = p(x|h^1)p(h^1|h^2)p(h^2|h^3)p(h^3) \tag{1}$$

  where $x$ is the actual observations, $h^1$ is low-level features, and $h^3$ is the top-most latent representation. $p(h^1|h^2), p(h^2|h^3)$ can then be viewed as priors over the latent representations or intermediate features.

  These model are easy to sample from (ancestral sampling) but hard to train since $p(x)$ is intractable.

**Remark 2** (Ancestral sampling)**.** Ancestral sampling is a method of sampling a probability graphical model by following the causal structure of the model. For example, we can sample $h^3$ from its prior distribution, then sample $h^2$ conditioned on $h^3$, and so on.

- **Undirected graphical models (Energy-based).** An example is Boltzmann machines. They rely on an energy function:

$$E(x, h^1, h^2, h^3) = -xW^1h^1 - h^1W^2h^2 - h^2W^3h^3 \tag{2}$$

  and define the joint probability as

$$p(x, h) = \frac{1}{Z}\exp(-E(x, h)) \tag{3}$$

  where $Z$ is the normalization constant, but also called partition function in this context.

  In these models, we can easily compute $p(x)$ up to a constant, but exact value of $p(x)$ is still nearly impossible to compute since the partition function $Z$ is often intractable to compute. Moreover, it is hard to sample from these models since we can't do ancestral sampling, and have to rely on Markov Chain Monte Carlo (MCMC) methods.

## 2 *Autoregressive generative models*

Autoregressive models treat data generation sequentially by

$$p(\mathbf{x}) = \prod_{i=1}^{n} p(x_i | x_1, \ldots, x_{i-1}) \tag{4}$$

Hence, they are most well-known for sequential data (language modeling, time series, etc.) and less applicable to non-sequential observations.

These includes MADE, Spatial LSTM, PixelRNN, PixelCNN, and WaveNet, etc. Although they're no longer the primary frameworks nowadays, they serves as a bridge to more modern generative models.

**Remark 3** (Not yet a latent variable model). Here, the previous observations $x_1, \ldots, x_{i-1}$ functions as surrogate of the latent representation $h^{(i-1)}$.

That said, these models have no latent variables and are not estimating a joint distribution of observation with latent variable $p(\mathbf{x}, \mathbf{h})$ explicitly. Thus, they are not latent variable models yet, but rather a fully observed model.

Nonetheless, they are indeed directed models since they rely on the causal direction of $x_1 \to x_2 \to \ldots \to x_n$.

### 2.1 *Masked autoencoders for distribution estimation (MADE)*

MADE is a recipe to create weight masks so that an autoencoder's reconstruction is autoregressive, i.e., the computation for predicting $x_i$ only depends on $x_1, \ldots, x_{i-1}$.

In image generation, it abandons the spatial structure of the image, and instead flatten the input and assigns an arbitrary random sequence to the pixels. During generation, it generates pixels one by one through that ordering. Each variable must be sampled one at a time, and then fed back into the network for the prediction of the next variable. Computing $p(x)$ only require one forward pass, however, when the sampling, we need $n$ forward passes to generate one image.

In practice, very large hidden layers may be required as not all hidden units are can contribute to each conditional. The training has the same complexity as a regular autoencoder.

The model is trained through a teacher-forcing approach, i.e., the model is trained to produce a high-quality reconstructed $\hat{\mathbf{x}}$ given the ground-truth $\mathbf{x}$ as input, instead of using the model's own predictions as input.

Stated mathematically, we have:

$$\hat{\mathbf{x}} = \text{decode}(\text{encode}(\mathbf{x})) = \text{MADE}(x) \tag{5}$$

$$\mathcal{L}(\mathbf{x}) = -\sum_{i=1}^{|\mathbf{x}|} (x_i \log \hat{x}_i + (1 - x_i) \log(1 - \hat{x}_i)) \tag{6}$$

the loss here is called the negative log-likelihood (NLL) loss for a binary variable.

The encode, decode here refer to encoding the input into the hidden layer activations, instead of the latent representation in a VAE.

### 2.2 *PixelCNN*

Similar to MADE, where we enforce the autoregressive relationship in autoencoders using masked weights, PixelCNN achieve the same

goal in convolutional neural networks (CNN) through masked convolutions.

Specifically, the model use a raster scan ordering (from top left to bottom right) together with masked convolutional kernel to ensure the directed causal structure of the model.

Notice that convolution can make this raster scan faster compared to sequential models.

It's easy to imagine that training (in teacher-forcing framework) can now be parallelized just like regular CNNs, though sampling still requires sequential operations over the pixels.

One immediate problem follows is limited size of the receptive field.

If we only have one layer, each generated pixel will only use the information of a small amount of previous pixels in the receptive field. To avoid this, we can stack multiple layers of masked convolution.

However, stacking layers of masked convolution creates fo a blind spot in the receptive field. The model then ignore parts of the legal information it can use when generating pixels. Introducing both vertical and horizontal stack of masked convolution can solve this problem.

Moreover, it also use a specialized nonlinear gating mechanism to control the information flow, especially between vertical and horizontal stack of masked convolution, which helps the model handel the highly nonlinear pixel distribution in natural images.

More details of how the two stack convolution can be found in this blog post.

Lastly, the model is also autoregressive on the color channels, i.e., it generates three color channels (R, G, B) sequentially and produces discrete output through a softmax layer.

## 2.3 *WaveNet*

WaveNet applied the idea of PixelCNN to 1D audio signals.

Because audio happens at a high frequency, the model needs to see far back in time. It uses "dilated convolutional structure" to exponentially grow the receptive field.

Instead of modeling raw sound linearly, it adopts a signal transformation to make their output better aligns with the non-linear way human ears perceive sound amplitudes.

Specifically, the transformation is called $\mu$-law companding transformation. This not actually how we perceive the sound, but it allowed them to accentuate the lower level variations that we are sensitive to.

WaveNet also has conditional generation, including global conditioning (e.g., speaker ID) and local conditional (e.g., text) (convolutions over the text itself).

Moreover, it can be parallelized via distillation training by matching the KL divergence between a fixed teacher model (the original WaveNet) and a student model (a non-autoregressive version of WaveNet).

Specifically, the student is trained to minimize $D_{KL}(p_{\text{Student}}||p_{\text{Teacher}})$ where we use the reverse KL divergence to avoid the mode collapse,

forcing the student to cover the entire distribution of the teacher instead of just finding one highly likely sequence.

After training, during inference, the student model transforms a random noise signal into desired audio in parallel.

Additional losses are used to improve performance, including:

- **Power loss:** matching the power spectrum to real data (speech)
- **Perceptual loss:** distance in pre-trained classifier activation space.
- **Contrastive loss:** bring the output closer to similar data and farther from dissimilar data.

## 3 *Normalizing flows (Flow-based generative models)*

The essence of normalizing flows is translating probability distributions. We aim to map a simple, tractable distribution (e.g., Gaussian) in latent space to the complex data distribution in observation space using a deterministic, parameterized, and invertible (bijective) mapping.

We will see later that, the deterministic and invertible properties of the mapping can be dropped in variational methods.

We first review the change of variable formula for probability density. Suppose we have a random variable $x$ with density $p(x)$, and another random variable $y = f(x)$ where $f$ is a bijective function.

If $f$ is monotonic, for $f : \mathbb{R} \to \mathbb{R}$, we have:

$$p(y) = \frac{d}{dy} P(Y \leq y) = \frac{d}{dy} P(f(X) \leq y) = \frac{d}{dy} P(X \leq f^{-1}(y)) \tag{7}$$

$$= p(f^{-1}(y)) \left| \frac{df^{-1}(y)}{dy} \right| \quad \text{if } y \in f(\mathbb{R}), \text{ else } 0 \tag{8}$$

Extending to the multivariate case, $f : \mathbb{R}^m \to \mathbb{R}^m$, we have:

$$p(y) = p(f^{-1}(y)) \left| \det \frac{df^{-1}(y)}{dy} \right| \quad \text{if } y \in f(\mathbb{R}^n), \text{ else } 0 \tag{9}$$

notice that the determinant $\left| \det \frac{df^{-1}(y)}{dy} \right|$ of the $m \times m$ Jacobian matrix usually requires $O(m^3)$ time to compute.

In practice, we are usually using the logarithm of the density, and hence the change of variable formula becomes:

$$\log p(y) = \log p(f^{-1}(y)) + \log \left| \det \frac{df^{-1}(y)}{dy} \right| \quad \text{if } y \in f(\mathbb{R}^n), \text{ else } -\infty \tag{10}$$

Hence, three constraints of the framework are:

- **Strict invertibility.** The mapping must be bijective.
- **No bottleneck.** The dimensionality of the latent space must be equal to the data space.
- **Tractable Jacobian determinant.** The determinant of an $N \times N$ Jacobian matrix usually requires $O(N^3)$ time to compute, which are intractable for high-dimensional data. Thus, the model must be designed such that their Jacobian determinant can be computed efficiently (e.g., determinant of a triangular matrix).

In practice, flow-based models can usually be categorized into two types based on whether they use residual connection or ordered partial dependency to design the scalable normalizing flow.

We now cover specific implementation of flow.

### 3.1 *Residual flows*

Residual flows are in the form

$$f(x) = x + g(x), \tag{11}$$

where $g$ is a specially designed learnable perturbation function appended to the identity mapping. The incredibility of the mapping $f(x)$ can be obtained as long as the perturbation $g(x)$ is not too large.

**Planar and Sylvester flows**

Planar and Sylvester flows are two ways of designing the perturbation function $g$, using properties of matrix determinant. Planar flow has the general form of

$$f(x) = x + uh(w^T x + b) \tag{12}$$

where $u, w \in \mathbb{R}^D$, $b \in \mathbb{R}$, and $h$ is a non-linear activation function.

The Jacobian determinant can be computed as:

$$\left|\det \frac{df(x)}{dx}\right| = \left|\det(I + h'(w^T x + b)uw^T)\right| \tag{13}$$

$$= \left|1 + h'(w^T x + b)w^T u\right| \tag{14}$$

Where the second equality comes from the matrix determinant lemma (Wiki):

$$\det(I + uv^T) = (1 + v^T u) \tag{15}$$

Given that $h = \tanh h$, the invertibility of the mapping can be guaranteed by the condition $w^T u > -1$.

Sylvester flow is a more complicated planar flow, it generalized the the 1-dimensional bottleneck in planar flow to $d$-dimension, it has a general form of:

$$f(x) = x + Ah(Bx + b) \tag{16}$$

where $A \in \mathbb{R}^{m \times d}$, $B \in \mathbb{R}^{d \times m}$, $b \in \mathbb{R}^d$, $d \leq m$, and $h$ is a non-linear activation function. The Jacobian determinant is computed using the Sylvester's theorem (Wiki). We would not go into the details of the condition here, but the argument is very similar to the planar flow.

Overall, because each single layer of planar or Sylvester flow are weak, limited by the condition for invertibility, we have to compose many of these simple invertible transformations.

**Stochastic estimation for general residual form**

For a general residual flow of the form $f(x) = x + g(x)$, it is invertible as long as $g$ is a contractive mapping:

**Theorem 1.** *If $g(x)$ is L-Lipschitz for $0 < L < 1$, then $f(x) = x + g(x)$ is invertible.*

Proof: Fix any $y$ in the codomain of $f$, defined $x_0 = 0$ and $x_t = y - g(x_{t-1})$ for $t \geq 1$, which is a contraction mapping. Thus, by the Banach fixed-point theorem, $x_t$ converges to a unique fixed point $x^*$ such that $f(x^*) = y$.

Thus, although for this general residual flow, we don't have a closed-form solution for the inverse mapping, we can recover the inverse through fixed-point iteration under the contractive assumption.

This allows us to have more flexible $g$ with greater capacity, e.g., a neural network. However, calculating the exact determinant of such transformation is difficult.

Recall that we usually look at the log-density and hence the log-determinant (eq. (10)). We can estimate the log-determinant in the following way:

$$\log \left|\det J_f(x)\right| = \operatorname{tr}\left(\log J_f(x)\right) \quad \text{matrix identity} \tag{17}$$

Taylor expansion of $\log(I + J_g(x))$:

$$\operatorname{tr}\left(\log\left(I + J_g(x)\right)\right) = \sum_{k=1}^{\infty} \frac{(-1)^{k+1}}{k} \operatorname{tr}\left(J_g(x)^k\right) \tag{18}$$

by Hutchinson's trace estimator, where $v$ is a random vector with zero mean and identity covariance matrix.

$$\operatorname{tr}(A) = \mathbb{E}_{p(v)}\left[v^T A v\right] \tag{19}$$

Let $A = \log\left(I + J_g(x)\right)$ and truncate the infinite series

$$\operatorname{tr}(\log\left(I + J_g(x)\right)) \approx \mathbb{E}_v\left[\sum_{k=1}^{n} \frac{(-1)^{k+1}}{k} v^T \left[J_g(x)\right]^k v\right] \tag{20}$$

we note that truncation introduce a bias.

## 3.2 *Partial dependency flow*

Early methods like residual flows do not scale well. By leveraging partial dependency, we can design more scalable method with complex transformations while still maintain invertibility. This structure forces the Jacobian to be triangular, making the determinant easy to compute (product of diagonal entries) even the transformation is complex and has high capacity.

In particular, this can be done by using **coupling flows**, which create a block-triangular Jacobian, or **autoregressive flows**, which create a triangular Jacobian.

**Coupling flows**

**Autoregressive flows**

## 3.3 *Continuous flows*

The methods we discussed in this section are all considered discrete normalizing flows, which use a finite sequence of invertible transformations to map between the latent and data space (represented by discrete number of layers in the model). Continuous normalizing flows (CNFs) model the transformation as a continuous-time dynamic system, through neural ordinary differential equations (ODEs). They have become increasingly popular in recent years.

Early methods include FFJORD are simulation-based and computationally expensive, they are followed by the flow matching frameworks, which solve the scalability issue by directly learning the vector field of the ODEs, allowing for simulation-free training.

Flow matching bridges the gap between normalizing flows and diffusion models. Reference for further reading.

## 3.4 *Summary and big picture*

# 4 *Variational autoencoders (VAEs)*

**Autoencoder as generative models**

We consider an autoencoder with some loss function. If we can recover the distribution $p(H)$ of the hidden units $H$, then sampling $H \sim p(H)$ and pushing it through the decoder will return the data distribution $p(X)$ in the observation space.

In fact, this construction is considered as a more general framework of generative modeling–latent variable models, where we learn a mapping from some latent variable $z$ to a complicated distribution on the data space $x$.

## *References*